Proyecto 1 — Rover autónomo

Búsqueda, planificación y metaheurísticas para la exploración de una región.

Autor: Angélica María Martínez Céspedez

Asignatura: Elementos de Inteligencia Artificial

Resumen

Se implementó un agente autónomo capaz de explorar una región marciana modelada como una rejilla con costos de terreno, recolectar y analizar muestras científicas y transmitirlas, sujeto a restricciones de batería, memoria y ubicación. El sistema integra cuatro módulos que se conectan entre sí: búsqueda (BFS, DFS, Dijkstra, A* y voraz), metaheurísticas (Hill Climbing y Recocido Simulado) para decidir el orden de visita de los POI, planificación simbólica con A* sobre un espacio abstracto de acciones con precondiciones, y un sistema basado en reglas (módulo opcional E). El generador de entornos es paramétrizable en tamaño, estructura y puntos de carga, y garantiza que toda instancia tenga solución.

La comparación experimental se hace con la métrica apropiada para un problema de rutas sobre instancias heterogéneas: para cada tramo se reporta el cociente entre el coste del algoritmo y el coste óptimo (o, en el caso de nodos expandidos, el cociente respecto a una referencia). Esto evita mezclar tramos de distinta longitud y permite interpretar la dispersión como variabilidad del algoritmo y no como diferencia entre escenarios distintos. En ese marco, A* obtiene la misma solución que Dijkstra en coste pero con un número claramente menor de nodos expandidos y un tiempo competitivo; las metaheurísticas se comparan frente al óptimo solo en los tamaños donde el problema todavía es calculable por enumeración, y para tamaños mayores se evalúan frente a un orden aleatorio con el mismo presupuesto de cómputo. La justificación del módulo de metaheurísticas no viene de la escala de la decisión del orden por sí sola, sino de la incapacidad del planificador exacto para seguir escalando cuando crece el número de POI.

1. Descripción y modelado del problema

El rover se mueve por una rejilla de celdas (`width × height`). Cada celda tiene un terreno con un costo de entrada: llanura (1), piedra (2), roca (3), arena fina (4) y duna (5); los obstáculos son intransitables. Sobre la rejilla se ubican cuatro tipos de puntos:

- Base: punto de recarga y de partida.
- Estaciones de carga: puntos de recarga adicionales (la base ya es uno de ellos).
- POI: las muestras científicas que hay que procesar.
- Zonas de comunicación: los únicos lugares desde donde se puede transmitir.

El rover tiene batería (capacidad 100, se agota con las acciones y se recarga gratis en cualquier punto de recarga) y memoria (capacidad 3 muestras pendientes). El estado del agente es la posición, la batería, la memoria usada y los conjuntos de muestras recolectadas, analizadas y transmitidas.

Las acciones son: `move` (desplazarse, consume batería igual al costo del terreno de destino), `collect` (10, ocupa un lugar de memoria), `analyze` (5, solo donde se recolectó), `transmit` (5, solo en una zona de comunicación, libera un lugar de memoria) y `recharge` (en la base o en una estación de carga, gratis, restaura la batería). La misión consiste en recolectar, analizar y transmitir todas las muestras. La restricción de memoria obliga a desviarse a transmitir cada tres muestras, y la batería obliga a planear las recargas; la combinación de ambas hace que el problema no sea un simple ruteo geográfico.

La puntuación de una misión es `100 · (muestras transmitidas) − costo total`, donde el costo total es la suma de los costos de las acciones.

Generador parametrizable y con solución garantizada. Las instancias se generan de forma reproducible a partir de una semilla y son parametrizables en tres ejes: tamaño (`width`, `height`), estructura (mezcla de terre-

nos y densidad de obstáculos) e infraestructura (`n_charging_stations`). Además, el generador garantiza que la misión sea resoluble: primero conecta todos los puntos clave y después asegura que cada punto clave sea alcanzable dentro de la capacidad de la batería desde algún punto de recarga. Así ninguna instancia es irresoluble por falta de energía.

Se usan dos representaciones complementarias: un grafo implícito de la rejilla (estados = celdas, acciones = 4 vecinos) para la búsqueda de rutas, y un grafo de puntos clave (base, estaciones de carga, POI y zonas de comunicación) cuyas distancias se precalculan con A* para la planificación y las metaheurísticas.

2. Módulo de búsqueda

Se implementaron cinco algoritmos sobre la rejilla: dos no informados — BFS y DFS —, Dijkstra, A* con la heurística de distancia Manhattan y una búsqueda voraz. Este módulo resuelve la ruta de menor costo entre dos puntos, y es la pieza que reutilizan la planificación y las metaheurísticas.

Protocolo experimental. Se evaluaron 5 instancias de 16×16 con 5 POI (semillas 11, 23, 37, 41, 59). En cada instancia se resolvieron 5 tramos, de forma que se compararon 25 tramos por algoritmo. Para evitar mezclar tramos de distinta longitud, la comparación se hace con un cociente por tramo:

- `q = costo_algoritmo / costo_optimo`
- `q_n = nodos_expandidos_algoritmo / nodos_expandidos_dijkstra`

La dispersión se reporta sobre esos cocientes, no sobre la suma de costos de tramos heterogéneos. Los tiempos se reportan también como media ± desviación sobre las mismas ejecuciones.

Algoritmo	Cociente de costo `q` vs Dijkstra	Cociente de nodos `q_n` vs Dijkstra	Tiempo (ms)
BFS	1,39 ± 0,28	1,19 ± 0,40	0,13 ± 0,04
DFS	10,07 ± 17,73	4,89 ± 9,77	0,16 ± 0,05
Dijkstra	1,00 ± 0,00	1,00 ± 0,00	0,17 ± 0,06
A*	1,00 ± 0,00	0,46 ± 0,18	0,10 ± 0,03
Voraz	1,44 ± 0,31	0,16 ± 0,10	0,03 ± 0,01

Análisis. Dijkstra y A* devuelven rutas del mismo coste (ambos óptimos), por eso su cociente de costo es exactamente `q = 1,00 ± 0,00`. La diferencia aparece en esfuerzo de búsqueda: A* usa `q_n = 0,46 ± 0,18`, es decir, expande en promedio ~54 % menos nodos que Dijkstra en comparación pareada por tramo. El algoritmo voraz es el más rápido y el que menos nodos recorre, pero paga un coste mayor. BFS y DFS confirman por qué hace falta la información del terreno: minimizan pasos o encuentran una ruta rápida, pero no optimizan el coste del terreno. En consecuencia, A* es la opción por defecto en el resto del sistema.

Comparación entre estructuras. Para comprobar que los algoritmos se comportan igual en sistemas distintos, se repitió la búsqueda con el mismo tamaño y número de POI pero con tres perfiles de estructura: `flat`, `default` y `rough`. El coste óptimo (A*) pasa de 10,4 (flat) a 14,3 (default) y a 18,2 (rough). Esto muestra que el coste real cambia con la dureza del terreno, pero el orden entre algoritmos se mantiene. Este análisis valida el generador parametrizable: las conclusiones no dependen de un solo mapa.

3. Módulo de metaheurísticas

Qué se decide y por qué. El planificador exacto sobre el espacio completo de estados se vuelve intratable al aumentar el número de POI: si hay `n` POI, el número de órdenes de visita posibles es `n!`. Por eso se aborda directamente la permutación de visita de los POI con metaheurísticas y se pasa el orden propuesto al planificador.

- Representación: una permutación de los POI.
- Vecindad: intercambio de dos posiciones.
- Función objetivo: coste estimado de recorrer los POI en ese orden respetando la capacidad de memoria, las

desviaciones a zonas de comunicación y el retorno a recarga. La función está alineada con el modelo del problema y refleja la estructura del plan real.

- Algoritmos: Hill Climbing (acepta el vecino solo si mejora) y Recocido Simulado (acepta empeoramientos con probabilidad decreciente). Ambos usan el mismo presupuesto de evaluaciones.

Protocolo experimental. Se generaron 5 instancias de 14×14 con 4 POI, y para cada instancia se ejecutaron 8 corridas por algoritmo, con semillas declaradas. La comparación se hace por instancia y por corrida: se reporta la media y la dispersión dentro de cada instancia, y la variabilidad global se interpreta como agregación de esa variación.

Algoritmo	Función objetivo	Coste del plan real	Tiempo (ms)
Hill Climbing	155,4 ± 13,7	168,9 ± 19,7	44,6 ± 12,1
Recocido Simulado	154,8 ± 13,3	168,9 ± 20,0	29,5 ± 8,2

Análisis. Con el mismo presupuesto, ambos algoritmos alcanzan una calidad descriptivamente muy parecida. Sin embargo, la formulación "estatísticamente equivalente" requiere una prueba estadística pareada sobre las mismas instancias y semillas (por ejemplo, Wilcoxon o t-test). Aquí se interpreta el resultado como "equivalencia práctica" (diferencia media ~0,4 puntos de función objetivo), pero no se reclama equivalencia estadística formal sin test. Además, la dispersión relevante para comparar estocásticos es la intra-instancia: para la función objetivo, HC tiene desviación intra-instancia media `±1,27` y SA `±0,00`; en costo de plan, HC `±2,60` y SA `±0,52`. El valor global `±13,x` queda dominado por heterogeneidad entre mapas, no por aleatoriedad del método. El Recocido Simulado es más rápido, y en este código la causa principal es operativa: SA corta antes de `2000` evaluaciones por el criterio de temperatura (`temp < 1e-4`), mientras HC consume siempre el presupuesto completo. La diferencia entre la función objetivo y el coste del plan real (~8–9%) se reporta explícitamente: la función objetivo omite algunas desviaciones extras por batería y recarga, así que subestima el coste final del plan.

Justificación del módulo. La justificación se hizo con mediciones concretas de escalamiento del problema de orden. En la evaluación exacta por fuerza bruta del orden (misma función objetivo), el tiempo medio fue: `n=4` → `0,51 ms`, `n=5` → `3,12 ms`, `n=6` → `20,79 ms`, `n=7` → `212,59 ms`, `n=8` → `2037,33 ms`, `n=9` → `22622,47 ms` (una sola instancia ya tarda ~22,6 s). La explosión factorial (`n!`) vuelve inviable enumerar a medida que crece `n`.

En el régimen donde sí se puede calcular exacto (hasta `n=6` en nuestras pruebas), la brecha media respecto al óptimo del orden fue: HC `0,00 %` (`n=4`), `0,32 %` (`n=5`) y `1,89 %` (`n=6`); SA `0,00 %` en `n=4,5,6` para las semillas evaluadas. En el benchmark de planificación, ambas metaheurísticas quedan a `+2,69 % ± 2,24 %` del óptimo del plan y mejoran claramente sobre el orden aleatorio (`+16,44 % ± 11,59 %`). Esta evidencia, y no una afirmación metodológica abstracta, es la que justifica el módulo D.

4. Módulo de planificación (representación simbólica elegida)

Se eligió planificación clásica en lugar de CSP. El estado del planificador es la tupla `(posición, batería, memoria usada, muestras recolectadas, analizadas, transmitidas)` y el espacio se explora con A* sobre acciones con precondiciones y efectos:

Acción	Precondición	Efecto	Costo
`move(d)`	batería ≥ costo del tramo	posición ← d; batería −= costo	terreno de `d`
`collect(p)`	estar en `p`, sin recolectar, memoria libre	recolectada; memoria +1	10
`analyze(p)`	estar en `p`, recolectada	analizada	5
`transmit(p)`	en zona de comunicación, `p` analizada	transmitida; memoria −1	5
`recharge`	en la base o en una estación de carga, batería < 100	batería ← 100	0

La meta es transmitir todas las muestras. Los movimientos entre puntos clave se resuelven con A* sobre la rejilla, lo que integra el módulo de búsqueda dentro del de planificación.

La heurística del planificador combina dos cotas inferiores: (i) las acciones aún obligatorias para cada muestra sin transmitir, y (ii) una cota de movimiento dada por el árbol de expansión mínimo sobre los POI pendientes y una zona de comunicación. Esta heurística es admisible y la implementación mantiene optimalidad porque actualiza `g(n)` y reinyecta estados en la frontera cuando encuentra un camino mejor (reapertura efectiva de nodos); no depende de una clausura rígida sin reapertura. En otras palabras, la optimalidad se entiende como optimalidad dentro del grafo abstracto de puntos clave, no como optimalidad global sobre toda la rejilla como un sistema continuo de estados físicos.

Integración con las metaheurísticas. El parámetro `enforce_order=True` restringe el orden de recolección al propuesto por la metaheurística o por las reglas: el planificador lo respeta, pero sigue decidiendo libremente cuándo recargar y cuándo transmitir. Sin este mecanismo, el planificador ignoraba el orden y las metaheurísticas no tenían efecto real sobre la solución.

Validación. Cada plan se ejecuta en un simulador que comprueba las precondiciones paso a paso: que no se agote la batería, que no se exceda la memoria, que se analice en el POI y que se transmita solo en una zona de comunicación. Los planes tienen entre 20 y 32 acciones; en la demostración se usa explícitamente la estación de carga.

Limitación. El planificador exacto no escala bien con el número de POI. Esta limitación es el argumento empírico para el módulo de metaheurísticas: cuando el espacio de órdenes se vuelve inmanejable, se necesita una aproximación.

5. Módulo adicional: sistema basado en reglas (módulo E, opcional)

Se implementó además un sistema de producción con encadenamiento hacia adelante, con reglas como “si hay una muestra recolectada sin analizar en la posición actual → analizar”; “si la memoria está llena → ir a la zona de comunicación más cercana y transmitir”; “si ningún destino es alcanzable con margen → ir al punto de recarga más cercano”; y “si quedan POI sin recolectar → ir al más cercano”. El módulo produce un plan completo sin búsqueda exhaustiva y es totalmente explicable.

Resolución de conflictos. Cuando se cumplen varias reglas a la vez, la política resuelve el conflicto con una jerarquía explícita de prioridades. En la implementación actual, el orden es:

```
recharge > transmit > analyze > collect > goto_charge > goto_comm > goto_poi
```

Esta prioridad se elige deliberadamente para asegurar la viabilidad del sistema: si la batería no tiene margen para volver a recargar, la recarga tiene precedencia sobre la recolección; si hay datos analizados pendientes de transmitir, se prefieren antes que seguir recolectando; y solo cuando la energía y la memoria están bajo control se activa la recolección y el movimiento hacia el siguiente POI. Así, la resolución de conflictos prioriza la seguridad energética y la completitud de la misión sobre la eficiencia local.

Instancia	Misiones completadas	Coste medio
12×12 / 4 POI	100 % (20/20)	183,55 ± 27,09
14×14 / 6 POI	100 % (20/20)	291,85 ± 43,39
16×16 / 5 POI	100 % (20/20)	279,65 ± 47,49

Las reglas son rápidas y completan la misión en el 100 % de las instancias, aunque tienen un coste mayor que el planificador exacto. En la demostración, el sistema de reglas da 217 frente a los 158 del óptimo. La ventaja está en la velocidad y la explicabilidad; el coste se paga como compensación por no resolver el problema globalmente.

6. Resultados experimentales y análisis comparativo

Protocolo (igual para todos los módulos). Se usan varias instancias (varios tamaños, varios números de POI y varias estructuras), varias corridas con semillas declaradas y se reporta media y desviación. La dispersión se separa en dos niveles:

- variación entre instancias: cambia el mapa, la estructura del terreno o la disposición de los POI;
- variación dentro de una instancia: distintas corridas con semillas distintas del algoritmo estocástico.

La comparación de algoritmos debe hacerse sobre el mismo conjunto de instancias y con la misma semilla o el mismo conjunto de réplicas. De no ser así, la desviación global refleja más la heterogeneidad de entornos que la variación del algoritmo.

Los hallazgos principales son:

(1) Búsqueda. A* obtiene la misma solución que Dijkstra con un menor número de nodos expandidos; BFS y DFS son peores porque ignoran el coste del terreno; el voraz es más rápido pero subóptimo.

(2) Estructura. El coste óptimo crece con la dureza del terreno, pero el orden entre algoritmos se mantiene. Esto muestra que la conclusión no depende de una única topología de mapa.

(3) Metaheurísticas. Hill Climbing y Recocido Simulado tienen un comportamiento descriptivamente muy parecido con el mismo presupuesto; la diferencia de tiempo favorece a SA, pero la diferencia debe interpretarse con cuidado y, si se quiere afirmar equivalencia estadística, exige prueba estadística pareada.

(4) Planificación frente a las metaheurísticas. Con el planificador exacto como referencia en instancias donde el óptimo puede calcularse:

Método	**Coste del plan**
Óptimo (A* exacto)	137,2 ± 12,1
Recocido Simulado + planificador	141,0 ± 14,0
Hill Climbing + planificador	141,0 ± 14,0
Orden aleatorio + planificador	160,4 ± 27,1

En esas instancias, las metaheurísticas quedan muy cerca del óptimo y muestran una mejora clara frente a un orden aleatorio. La diferencia se debe únicamente a la calidad del orden, porque el planificador exacto es el mismo en todos los casos.

(5) Demostración integrada (semilla 23, 12×12, 4 POI, 1 estación de carga). Hill Climbing y Recocido Simulado coinciden en el orden de visita; al imponerlo, el plan forzado cuesta 162 frente a los 158 del óptimo libre, una diferencia del 2,5 %. El planificador usa la estación de carga para recargar a mitad de misión. El sistema de reglas gasta 217, pero también completa la misión y lo hace de forma explicable y rápida.

(6) Comparación con el óptimo en las mismas instancias. La comparación con el óptimo no debe reducirse a una instancia de demostración. La forma correcta es evaluar la brecha media en las mismas instancias en las que el óptimo es calculable y reportar la distribución del error relativo:

```
gap = (costo_metodo - costo_optimo) / costo_optimo
```

Aplicado al sistema de reglas en el mismo conjunto de instancias `12x12` con `4 POI`, se obtiene: `gap_reglas = (183,55 - 137,20) / 137,20 = +33,78 %`. Con esto se cuantifica explícitamente el costo de una política reactiva explicable frente a la optimización global.

7. Desafíos encontrados y cómo se resolvieron

- La metaheurística no influía en el plan: la primera versión del planificador recalculaba el mejor orden y

siempre devolvía la misma solución. Solución: añadir `enforce_order`, que restringe la recolección al orden propuesto.

- La función objetivo estaba desalineada: solo sumaba distancias e ignoraba la memoria y la batería. Solución:

alinearla con el modelo del problema (desviaciones por memoria, distancias reales de A* y acciones obligatorias).

- Presupuesto desigual: Hill Climbing hacía menos evaluaciones que SA. Solución: usar el mismo presupuesto para ambos algoritmos.

- Tableros sin solución: algunos POI quedaban encerrados o demasiado lejos para ser viables. Solución: el

generador del entorno garantiza conectividad y viabilidad energética.

- Rendimiento del planificador exacto: con 5–6 POI el A* podía tardar minutos. Solución: mantener el modelo exacto

en la abstracción elegida y justificar el uso de metaheurísticas para tamaños donde el problema ya no es tratable por enumeración.

- Rigor experimental: se fijaron semillas, se ejecutan varias corridas y la comparación se expresa en términos de

medias y dispersiones por instancia y por tramo, en lugar de sobre conjuntos heterogéneos mezclados.

8. Conclusiones y posibles mejoras

Conclusiones. A* es la opción más conveniente para la búsqueda de rutas (óptima en la abstracción elegida y más eficiente que Dijkstra). Las búsquedas no informadas y el algoritmo voraz quedan por detrás. Hill Climbing y SA encuentran órdenes de visita muy cercanos al óptimo cuando este es calculable y superan a un orden aleatorio con el mismo presupuesto. El planificador simbólico es correcto y óptimo en la abstracción escogida, pero su coste crece muy rápido con el número de POI. El generador parametrizable da una base sólida para comparar algoritmos en distintos sistemas sin perder reproducibilidad.

Mejoras posibles. (i) incluir una prueba estadística pareada sobre la misma familia de instancias; (ii) ampliar la comparación con el óptimo para más tamaños y reportar la brecha media; (iii) comparar más metaheurísticas (búsqueda tabú, AG, colonia de hormigas); y (iv) estudiar mejor la eficiencia de SA frente a HC cuando el número de evaluaciones es fijo.

Referencias

- Russell, S.; Norvig, P. *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson, 2021.
- Hart, P.; Nilsson, N.; Raphael, B. *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*.

IEEE Transactions on Systems Science and Cybernetics, 1968.

- Kirkpatrick, S.; Gelatt, C.; Vecchi, M. *Optimization by Simulated Annealing*. Science, 1983.